UNIT 2

Intelligent Agents and Problem Solving

Topics Covered:

Introduction to Agents and Environments • Nature of Environments and Agents • Problem-Solving Agents • Search Algorithms (Brute-Force, DFS, BFS) • Heuristic Search (Hill Climbing, Best-First Search, A*, AO*) • Knowledge Representation

1. Introduction to Agents

Definition: Agent

An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators. Formally, an agent's behaviour is described by the agent function, which maps any given percept sequence to an action: f : P* → A, where P* is the set of all possible percept sequences and A is the set of actions.

In simple terms, an agent continuously performs a Perceive → Think/Decide → Act cycle. A human agent uses eyes, ears and other organs as sensors, and hands, legs, mouth as actuators. A robotic agent uses cameras and infrared range finders as sensors, and motors as actuators. A software agent uses keystrokes, file contents and network packets as sensory input, and displays output or sends network packets as actions.

1.1 Agent Function vs Agent Program

- Agent Function: An abstract mathematical description mapping percept sequences to actions.
- Agent Program: The concrete implementation of the agent function that runs on the physical architecture (hardware/software).

Relationship: Agent = Architecture + Program. The architecture is the machine on which the program runs (a computer, robot body, etc.), while the program is the actual algorithm/code that implements the agent function.

1.2 Rational Agent

Definition: Rational Agent

A rational agent is one that, for each possible percept sequence, selects an action that is expected to maximise its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Rationality depends on four factors, often remembered by the acronym PEAS:

- Performance measure — the criterion of success (e.g., cleanliness, safety, efficiency).
- Environment — the surroundings the agent operates in.
- Actuators — the mechanisms through which the agent acts.
- Sensors — the mechanisms through which the agent perceives.

Example — PEAS for an Automated Taxi Driver

Performance measure: Safety, speed, legality, comfortable trip, profit.

Environment: Roads, other traffic, pedestrians, customers.

Actuators: Steering wheel, accelerator, brake, signal, horn, display.

Sensors: Cameras, GPS, speedometer, sonar, engine sensors, keyboard.

1.3 Applications of Intelligent Agents

- Autonomous vehicles and self-driving cars

- Personal digital assistants (Siri, Alexa, Google Assistant)
- Automated trading and stock-market bots
- Robotic vacuum cleaners (e.g., Roomba)
- Game-playing agents (chess, Go engines)
- Spam-filtering agents in e-mail systems
- Industrial and warehouse robots

2. Environments

Definition: Environment

The environment is everything external to the agent — the world in which the agent exists and operates, from which it receives percepts and upon which it acts through its actuators.

2.1 Properties / Nature of Task Environments

Task environments are characterised along several dimensions. These dimensions determine the design of the appropriate agent.

(a) Fully Observable vs Partially Observable

An environment is fully observable if the agent's sensors give it access to the complete state of the environment at each point in time; otherwise it is partially observable.

Example

Chess is fully observable (the board is completely visible). Poker is partially observable (opponents' cards are hidden).

(b) Deterministic vs Stochastic

If the next state of the environment is completely determined by the current state and the action executed by the agent, the environment is deterministic; otherwise it is stochastic.

Example

Chess is deterministic. Self-driving car navigation on real traffic roads is stochastic because of unpredictable behaviour of other vehicles.

(c) Episodic vs Sequential

In an episodic environment, the agent's experience is divided into atomic episodes; each episode consists of the agent perceiving and then performing a single action, and the next episode does not depend on previous ones. In a sequential environment, current decisions can affect all future decisions.

Example

Defect detection on an assembly line (each item independent) is episodic. Chess playing (each move affects future moves) is sequential.

(d) Static vs Dynamic

If the environment can change while the agent is deliberating, it is dynamic; otherwise it is static. A semi-dynamic environment is one where the environment itself does not change but the agent's performance score does with the passage of time.

> **Example**
> Crossword puzzle solving is static. Taxi driving in traffic is dynamic.

(e) Discrete vs Continuous

This distinction applies to the state of the environment, the way time is handled, and to percepts and actions. Chess has a finite number of distinct states and actions (discrete). Taxi driving involves continuous states (speed, location) and continuous time.

(f) Single-agent vs Multi-agent

An environment with only one agent acting is single-agent (e.g., solving a crossword alone). An environment with more than one agent, which may be cooperative or competitive, is multi-agent (e.g., chess is a competitive multi-agent environment).

(g) Known vs Unknown

In a known environment, the outcomes (or outcome probabilities) for all actions are given to the agent in advance. In an unknown environment, the agent must learn how the environment works in order to make good decisions.

2.2 Summary Table of Environment Types

Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Chess (with clock)	Fully	Deterministic	Sequential	Semi-dynamic	Discrete	Multi
Poker	Partially	Stochastic	Sequential	Static	Discrete	Multi
Taxi driving	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi
Crossword puzzle	Fully	Deterministic	Sequential	Static	Discrete	Single
Image analysis	Fully	Deterministic	Episodic	Semi-dynamic	Continuous	Single
Part-picking robot	Partially	Stochastic	Episodic	Dynamic	Continuous	Single

3. Types / Structure of Agents

Agents can be grouped into five classes based on their degree of perceived intelligence and capability, in order of increasing generality and complexity.

3.1 Simple Reflex Agent

Definition: Simple Reflex Agent

An agent that selects actions based only on the current percept, ignoring the rest of the percept history, using simple condition–action (if–then) rules.

Working: percept → condition-action rule lookup → action. It uses 'if condition then action' rules, e.g., IF car-in-front-is-braking THEN initiate-braking.

Example / Application

A thermostat that turns on the heater IF temperature < set point. A simple vacuum-cleaner agent that sucks dirt IF the current square is dirty, else moves to the next square.

Limitation: Works only if the environment is fully observable; it can easily get stuck in infinite loops in partially observable environments and cannot handle environments requiring memory of past states.

3.2 Model-Based Reflex Agent

Definition: Model-Based Reflex Agent

An agent that maintains some internal state that depends on the percept history and reflects at least some of the unobserved aspects of the current state, using a 'model' of how the world works (how the world evolves and how the agent's actions affect it).

It keeps track of the part of the world it cannot currently see by maintaining an internal state, updated using: (i) how the world evolves independently, and (ii) how the agent's own actions affect the world.

Example / Application

A self-driving car keeping track of other vehicles' positions even when momentarily out of camera view; a robotic vacuum remembering which rooms have already been cleaned.

3.3 Goal-Based Agent

Definition: Goal-Based Agent

An agent that, in addition to state information, has goal information describing desirable situations, and chooses actions that will (eventually) lead to achieving its goals — often requiring search and planning.

Unlike reflex agents, goal-based agents consider the future: 'What will happen if I do action A?' and 'Will that make me happy (achieve the goal)?' This makes them more flexible since the knowledge that supports decisions is represented explicitly and can be modified.

Example / Application

A GPS navigation agent that plans a route to a destination (goal) by searching among alternative paths; a chess-playing agent aiming for checkmate.

3.4 Utility-Based Agent

Definition: Utility-Based Agent

An agent that uses a utility function to map a state (or sequence of states) onto a real number describing the associated degree of happiness/goodness, enabling rational choice when there are multiple ways to achieve a goal or when goals conflict.

Goals alone are not always enough to generate high-quality behaviour — many action sequences reach the goal, but some are better (faster, safer, cheaper) than others. A utility function ranks states by desirability, allowing the agent to choose among conflicting goals and to weigh the likelihood of success against the importance of goals when outcomes are uncertain.

Example / Application

A ride-hailing routing agent that balances trip time, fuel cost and safety, choosing the path with maximum overall utility rather than merely 'any path that reaches the destination.'

3.5 Learning Agent

Definition: Learning Agent

An agent capable of learning from its past experience/percept history to improve its performance over time, rather than relying solely on built-in, fixed knowledge.

A learning agent has four conceptual components:

- Learning element — responsible for making improvements based on feedback.
- Performance element — responsible for selecting external actions (this corresponds to the whole agent seen earlier).
- Critic — provides feedback on how well the agent is performing, measured against a fixed performance standard.
- Problem generator — suggests exploratory actions that may lead to new, informative experiences.

Example / Application

A spam-filter agent that improves its classification accuracy as the user marks e-mails as spam/not-spam; recommendation systems (Netflix, Amazon) that refine suggestions based on user feedback.

3.6 Comparison of Agent Types

Agent Type	Uses Memory?	Uses Goals?	Uses Utility?	Learns?	Complexity
Simple Reflex	No	No	No	No	Lowest
Model-Based Reflex	Yes	No	No	No	Low
Goal-Based	Yes	Yes	No	No	Medium
Utility-Based	Yes	Yes	Yes	No	High
Learning Agent	Yes	Yes	Yes	Yes	Highest

4. Problem-Solving Agent

Definition: Problem-Solving Agent

A kind of goal-based agent that decides what to do by finding a sequence of actions that leads to a desirable/goal state; it formulates a problem, then applies a search algorithm to solve it before executing the resulting solution.

4.1 Steps in Problem Solving

1. Goal Formulation — deciding which aspects of the world are relevant (identifying what the agent wants to achieve), based on the current situation and the agent's performance measure.
2. Problem Formulation — deciding what actions and states to consider, given the goal (abstracting away irrelevant detail).
3. Search — the process of looking for a sequence of actions that reaches the goal; the output is a solution.
4. Execution — the agent carries out the actions recommended by the solution, one at a time.

4.2 Components of a Well-Defined Problem

- Initial State — the state the agent starts in.
- Actions — description of the possible actions available to the agent in a given state; ACTIONS(s) returns the set of actions that can be executed in state s.
- Transition Model — description of what each action does; RESULT(s, a) returns the state resulting from doing action a in state s.
- Goal Test — determines whether a given state is a goal state.
- Path Cost — a function that assigns a numeric cost to each path (sequence of state–action pairs); usually the sum of step costs.
- State Space — the set of all states reachable from the initial state by any sequence of actions, often visualised as a directed graph in which nodes are states and links are actions.
- Solution — an action sequence (path) from the initial state to a goal state; an optimal solution has the lowest path cost among all solutions.

4.3 Example Problems

(a) The 8-Puzzle Problem

8-Puzzle

States: A description of the location of each of the 8 tiles and the blank in one of the 9 squares.
Initial state: Any given arrangement of tiles.
Actions: Move blank Left, Right, Up, Down.
Transition model: Given a state and an action, returns the resulting state.
Goal test: Whether the state matches the goal configuration (tiles in numerical order).
Path cost: Each move costs 1, so path cost = number of moves.

(b) Route-Finding Problem (Romania Map Example)

Route Finding — Bucharest Example

A classic AI example (from Russell & Norvig): An agent in Arad needs to reach Bucharest.
States: Each city on the map.
Initial state: 'In Arad.'
Actions: Drive from the current city to a neighbouring city.
Transition model: RESULT(In(Arad), Go(Zerind)) = In(Zerind).
Goal test: 'Is the current city Bucharest?'
Path cost: Sum of road distances travelled.

(c) Other Classic Toy Problems

- Vacuum-cleaner world
- N-Queens problem
- Water-jug problem
- Missionaries and Cannibals problem
- Travelling Salesman Problem (TSP) — a real-world application in logistics and circuit design

4.4 Applications of Problem-Solving Agents

- GPS route planning and navigation systems
- Robot motion planning
- Puzzle-solving and game AI
- VLSI layout and circuit design
- Automated assembly-line sequencing
- Protein design / computational biology

5. Search Algorithms: Brute-Force (Uninformed) Search

Definition: Uninformed / Brute-Force Search

Search strategies that have no additional information about states beyond that provided in the problem definition — i.e., they do not know how close a given state is to the goal. They can only generate successors and distinguish a goal state from a non-goal state, exploring the state space blindly/systematically.

Brute-force search techniques evaluate every possible sequence of actions in a systematic manner (level-by-level or branch-by-branch) until a goal is found. They are simple to implement but can be computationally expensive.

5.1 Depth-First Search (DFS)

Definition: Depth-First Search

An uninformed search strategy that always expands the deepest unexpanded node in the current search tree first, backtracking to the most recent unexplored branch when a dead end is reached. It uses a LIFO (Last-In-First-Out) stack as its frontier data structure.

```
function DFS(problem):
    frontier ← Stack containing the initial node
    explored ← empty set
    while frontier is not empty:
        node ← frontier.pop()
        if GOAL-TEST(node.state): return SOLUTION(node)
        add node.state to explored
        for each action in ACTIONS(node.state):
            child ← CHILD-NODE(problem, node, action)
            if child.state not in explored and not in frontier:
                frontier.push(child)
    return failure
```

Properties:

Property	Value	Remarks
Completeness	No (Yes for finite state spaces)	Can loop forever down an infinite path
Optimality	No	May find a longer path before a shorter one
Time Complexity	O(b^m)	b = branching factor, m = maximum depth
Space Complexity	O(bm)	Only needs to store a single path plus siblings — very memory-efficient

Application

Maze solving, puzzle games, topological sorting, detecting cycles in a graph, solving Sudoku via backtracking, and exploring file/directory trees.

5.2 Breadth-First Search (BFS)

Definition: Breadth-First Search

An uninformed search strategy in which the root node is expanded first, then all successors of the root are expanded, then their successors, and so on — expanding the shallowest unexpanded node first. It uses a FIFO (First-In-First-Out) queue as its frontier.

```
function BFS(problem):
    node ← initial node
    if GOAL-TEST(node.state): return SOLUTION(node)
    frontier ← FIFO Queue containing node
    explored ← empty set
    while frontier is not empty:
        node ← frontier.dequeue()
        add node.state to explored
        for each action in ACTIONS(node.state):
            child ← CHILD-NODE(problem, node, action)
            if child.state not in explored and not in frontier:
                if GOAL-TEST(child.state): return SOLUTION(child)
                frontier.enqueue(child)
    return failure
```

Property	Value	Remarks
Completeness	Yes	Always finds a solution if one exists (finite b)
Optimality	Yes*	Optimal if all step costs are equal (e.g., cost = 1)
Time Complexity	O(b^d)	b = branching factor, d = depth of shallowest solution
Space Complexity	O(b^d)	Must store every node generated — very memory-intensive

Application

Shortest-path finding in unweighted graphs, peer-to-peer networking (finding nearby nodes), web crawlers, GPS navigation (unweighted case), and social-network friend-suggestion (finding people within k connections).

5.3 DFS vs BFS — Comparison

Criterion	DFS	BFS
Data structure	Stack (LIFO)	Queue (FIFO)
Memory usage	Low — O(bm)	High — O(b^d)
Completeness	Not guaranteed (infinite spaces)	Guaranteed (finite branching)
Optimality	Not optimal	Optimal for uniform step costs
Best suited when	Solution is deep / memory limited	Solution is shallow / shortest path needed

6. Heuristic (Informed) Search Algorithms

Definition: Heuristic Search

Search strategies that use problem-specific knowledge, beyond the definition of the problem itself, in the form of a heuristic function h(n) to estimate how close a state n is to the goal — allowing the search to find solutions more efficiently than uninformed search.

Definition: Heuristic Function h(n)

A function that estimates the cost of the cheapest path from state/node n to a goal state. h(goal) = 0. A heuristic is admissible if it never overestimates the true cost to reach the goal, and consistent (monotonic) if, for every node n and successor n' generated by action a, h(n) ≤ c(n, a, n') + h(n').

6.1 Hill Climbing Algorithm

Definition: Hill Climbing

A local-search, iterative-improvement optimisation technique that starts with an arbitrary solution and continually moves in the direction of increasing value (steepest/greatest improvement) — i.e., it moves to the neighbouring state with the best heuristic value — stopping when no neighbour has a better value ('climbing the hill' of the objective function).

It is often called 'greedy local search' because it grabs the best immediate neighbour without looking ahead or keeping track of a search tree — it uses very little memory (only the current state) and does not backtrack.

```
function HILL-CLIMBING(problem):
    current ← MAKE-NODE(problem.INITIAL-STATE)
    loop do:
        neighbor ← the highest-valued successor of current
        if VALUE(neighbor) ≤ VALUE(current):
            return current.STATE       // reached a peak
        current ← neighbor
```

Variants of Hill Climbing

- Simple Hill Climbing — evaluates one neighbour at a time and moves to the first neighbour that is better than the current state.
- Steepest-Ascent Hill Climbing — examines all neighbours and moves to the one with the greatest improvement (the 'best' successor), rather than the first improving one.
- Stochastic Hill Climbing — chooses randomly among the uphill neighbours, with the probability of selection possibly varying with the steepness of the uphill move; converges more slowly but sometimes finds better solutions.
- Random-Restart Hill Climbing — conducts a series of hill-climbing searches from randomly generated initial states, stopping when a goal is found; escapes local maxima by trying many starting points.

Problems with Hill Climbing

- Local Maximum — a peak that is higher than each of its neighbouring states but lower than the global maximum; the algorithm gets stuck here and cannot proceed further.
- Plateau — a flat area of the state-space landscape where all neighbours have the same value, so the algorithm cannot determine which direction is best (can lead to a random, undirected walk).
- Ridge — a sequence of local maxima that is very difficult to navigate because the orientation of the ridge compared to the available moves makes it look like every move leads downhill, even though the ridge itself leads to a higher overall point.

Example / Application

The 8-queens problem (placing 8 queens so none attack each other) solved by minimising the number of attacking pairs; the Travelling Salesman Problem (approximate solutions); VLSI circuit design and layout optimisation; job-shop scheduling; and neural-network weight tuning (gradient-based variants).

6.2 Best-First Search

Definition: Best-First Search

A generic informed search strategy in which a node is selected for expansion based on an evaluation function f(n); the node with the lowest (or highest, depending on convention) evaluation is expanded first, using a priority queue as the frontier. Greedy Best-First Search is the special case where f(n) = h(n), i.e., it expands the node that appears to be closest to the goal.

```
function GREEDY-BEST-FIRST-SEARCH(problem, h):
    frontier ← Priority Queue ordered by h(n), containing initial node
    explored ← empty set
    while frontier is not empty:
        node ← frontier.pop()   // node with lowest h(n)
        if GOAL-TEST(node.state): return SOLUTION(node)
        add node.state to explored
        for each action in ACTIONS(node.state):
            child ← CHILD-NODE(problem, node, action)
            if child.state not in explored/frontier:
                frontier.push(child, priority = h(child.state))
    return failure
```

Property	Value	Remarks
Completeness	No	Can get stuck in loops (like DFS) in some cases
Optimality	No	Greedy — ignores cost already incurred, g(n)
Time Complexity	O(b^m)	Worst case; but often much faster in practice with a good h
Space Complexity	O(b^m)	Must keep all generated nodes in memory

Application

Path finding in video games and robotics where a fast, 'good-enough' route is preferred over guaranteed shortest path; used as a building block in many navigation systems when speed matters more than optimality.

6.3 A* Algorithm

Definition: A* Search

A best-first search algorithm that evaluates nodes using f(n) = g(n) + h(n), where g(n) is the exact cost of the path from the start node to n, and h(n) is the estimated (heuristic) cost from n to the goal. A* is complete and optimal provided the heuristic h(n) is admissible (for Tree-Search) or consistent (for Graph-Search).

```
function A*-SEARCH(problem, h):
    frontier ← Priority Queue ordered by f(n) = g(n) + h(n), containing initial
node (g=0)
    explored ← empty set
    while frontier is not empty:
        node ← frontier.pop()   // node with lowest f(n)
        if GOAL-TEST(node.state): return SOLUTION(node)
        add node.state to explored
```

```
        for each action in ACTIONS(node.state):
            child ← CHILD-NODE(problem, node, action)
            g(child) ← g(node) + STEP-COST(node, action, child)
            f(child) ← g(child) + h(child.state)
            if child.state not in explored/frontier, or found with lower f:
                frontier.push(child, priority = f(child))
    return failure
```

Why A* Works: A* balances the cost already spent (g) against the estimated cost remaining (h). Since g grows as the search proceeds while h estimates what's left, f(n) estimates the total cost of the cheapest solution through n. If h(n) is admissible, A* using Tree-Search is optimal; if h(n) is also consistent, A* using Graph-Search is optimal and never needs to re-expand a node.

Property	Value	Remarks
Completeness	Yes	Given finite branching factor and positive step costs
Optimality	Yes	If h(n) is admissible (Tree-Search) / consistent (Graph-Search)
Time Complexity	Exponential (worst case)	Depends heavily on the quality/accuracy of h(n)
Space Complexity	O(b^d)	Keeps all nodes in memory — its main drawback

Example

Finding the shortest driving route from Arad to Bucharest, using straight-line distance to Bucharest as h(n): f(n) = (actual road distance so far) + (straight-line distance remaining). A* explores far fewer cities than BFS/DFS because the heuristic focuses the search toward Bucharest.

Applications

GPS and map navigation (Google Maps style routing); pathfinding for game characters and NPCs; robotics motion planning; network routing protocols; puzzle solvers (8-puzzle, 15-puzzle); natural language parsing.

6.4 AO* Algorithm

Definition: AO* Algorithm

'AO' stands for AND–OR; AO* is a best-first search technique used to search AND–OR graphs/trees, i.e., problems that can be broken down into a set of sub-problems where solving ALL sub-problems in an AND-set is required, whereas solving ANY ONE sub-problem in an OR-set suffices. Unlike A*, which searches OR graphs only, AO* is designed for problem decomposition (problem-reduction) representations.

In an AND-OR graph: OR nodes represent alternative ways to solve a problem (only one branch needs to succeed), while AND nodes represent a problem that has been decomposed into sub-problems, all of which must be solved (all branches must succeed). AO* uses the heuristic evaluation function to decide which node to expand, and it revises cost estimates backward (up the graph) as new information is discovered — a process called cost revision or backward propagation.

AO* Algorithm Steps

5. Start with the initial node; compute its heuristic value.
6. Traverse the graph starting at the initial node, following the current best (marked) path, selecting the successor with the minimum cost at each OR node, and expanding ALL successors at an AND node.
7. Select any non-terminal (unexpanded) leaf node from the current best path and expand it, generating its successors and computing their heuristic values.
8. Change the value of the newly expanded node to reflect the values of its successors — compute f'(n) using the AND/OR combination rule (sum of costs for AND nodes; minimum cost for OR nodes).
9. Propagate the change in cost backward up the graph to update the values of the ancestor nodes, and re-mark the best path (this may cause the algorithm to switch to a different branch than before).
10. Repeat steps 3–5 until the start node is 'solved' (all nodes along the marked path are terminal/goal nodes) or is found to be unsolvable.

Worked Example (conceptual)

Consider planning a trip that requires EITHER 'Book Flight' OR 'Book Train' (OR node — only one needed), and after arriving, the traveller must BOTH 'Book Hotel' AND 'Rent Car' (AND node — both required). AO* would explore the cheaper of Flight/Train (OR), then require the combined cost of Hotel + Car (AND) to be added, and would revise its choice if new cost information (e.g., a hotel price increase) changes which top-level branch is cheapest overall.

Applications

Game-tree analysis with composite sub-goals; expert systems and diagnostic reasoning (where a diagnosis may require confirming several symptoms together — AND — or any one of several alternative tests — OR); STRIPS-style AI planning problems that are naturally decomposed into sub-goals; natural-language understanding where a sentence's meaning may require parsing multiple clauses (AND) or choosing between alternative interpretations (OR).

6.5 A* vs AO* — Comparison

Criterion	A* Algorithm	AO* Algorithm
Graph type	OR graph only	AND-OR graph
Backtracking	Yes, can reconsider any node	No explicit backtracking; uses cost revision instead
Path found	A single path from start to goal	A solution sub-tree/sub-graph (may include several branches)
Use case	Single-path shortest-route problems	Problem-reduction / sub-goal decomposition problems

7. Knowledge Representation (KR)

Definition: Knowledge Representation

The area of Artificial Intelligence concerned with how knowledge about the world can be represented in a formal, symbolic way inside a computer, so that an intelligent agent (or knowledge-based system) can use that knowledge to reason, draw inferences, and solve complex problems, such as diagnosing illness or conversing in natural language.

A knowledge-based agent needs a knowledge base (KB) — a set of sentences expressed in a knowledge representation language — and an inference mechanism to derive new facts from existing ones.

7.1 Types of Knowledge

- Declarative Knowledge — passive knowledge in the form of facts stored in a database, describing 'what is true' (e.g., facts, objects, concepts).
- Procedural Knowledge — knowledge responsible for knowing 'how to do something' (e.g., rules, strategies, agendas, procedures).
- Meta-Knowledge — knowledge about other types of knowledge, i.e., 'knowledge about what we know.'
- Heuristic Knowledge — knowledge representing 'rules of thumb' based on prior experiences, useful in guiding reasoning.
- Structural Knowledge — describes relationships between concepts and objects, such as sets, hierarchies, part-whole relations.

7.2 Approaches / Techniques of Knowledge Representation

(a) Logical Representation

Uses formal logic (Propositional Logic and Predicate/First-Order Logic) with well-defined syntax and semantics to represent facts and derive conclusions using inference rules such as Modus Ponens.

Example

Propositional logic: 'It is raining' → Rain = True.

First-order logic: ∀x Human(x) → Mortal(x) (All humans are mortal), combined with Human(Socrates) allows the inference Mortal(Socrates).

(b) Semantic Network

Definition: Semantic Network

A graphical knowledge-representation technique that represents knowledge as a network of nodes (representing objects, concepts, or situations) interconnected by labelled, directed arcs (representing relationships such as 'is-a' or 'has-a') between those objects.

Example

Node 'Sparrow' —is-a→ Node 'Bird' —is-a→ Node 'Animal'; Node 'Bird' —has-property→ 'Can Fly'; because Sparrow is-a Bird, it inherits the property 'Can Fly' (property inheritance).

(c) Frame Representation

Definition: Frame

A record-like structure that divides knowledge into substructures by representing 'slots' (attributes) and their corresponding 'values' for a particular object or concept, similar to an object in object-oriented programming.

Example

Frame: 'Car' — Slots: Colour = Red, Model = Sedan, Wheels = 4, Owner = Ravi. Frames can also contain default values and procedural attachments (e.g., a slot value computed by a small procedure).

(d) Production Rules (Rule-Based Representation)

Definition: Production Rule

A knowledge-representation format consisting of an IF (condition/antecedent) – THEN (action/consequent) pair, used extensively in expert systems; the production system also includes a working memory (facts) and a recognise–act cycle (the inference engine that matches rules against the working memory and fires applicable rules).

Example

IF the patient has a fever AND a sore throat AND a positive strep test, THEN diagnose Streptococcal pharyngitis. Rule-based expert systems such as MYCIN (medical diagnosis) use thousands of such production rules.

(e) Ontologies

A more modern, formal approach where concepts, categories and their relationships within a domain are explicitly specified (often using languages like OWL), enabling shared and reusable knowledge across different systems — widely used in the Semantic Web.

7.3 Requirements of a Good Knowledge Representation System

- Representational Adequacy — the ability to represent all the kinds of knowledge needed in that domain.
- Inferential Adequacy — the ability to manipulate the represented knowledge to derive new knowledge.
- Inferential Efficiency — the ability to incorporate additional information into the knowledge structure that can help focus the attention of the inference mechanisms in the most promising directions.
- Acquisitional Efficiency — the ability to acquire new knowledge easily, using automatic methods wherever possible rather than relying solely on human intervention.

7.4 Applications of Knowledge Representation

- Expert systems (e.g., MYCIN for medical diagnosis, DENDRAL for chemical analysis)
- Natural language processing and understanding (chatbots, virtual assistants)
- Semantic Web and the linking of structured data (ontologies, knowledge graphs like Google's Knowledge Graph)
- Robotics — representing spatial and task knowledge for planning
- Automated reasoning and theorem proving
- Recommendation systems that reason over user-preference knowledge

— End of Unit 2 Notes —